

UNIVERSIDADE CATÓLICA DE BRASÍLIA

Inteligência Artificial

CLASSIFICAÇÃO DE RISCO DE CRÉDITO:

Uma Comparação entre Algoritmos de Machine Learning

em R e Python

Relatório de Ciência de Dados

Integrante 1:	Ana Eduarda Raposo Medeiros - UC23200232
Integrante 2:	Anette Stefany Villalba Palomino - UC23200441
Integrante 3:	Clarice Christine Soares Viana - UC23200796
Professor(a):	William Malvezzi
Disciplina:	Inteligência Artificial
Data de entrega:	25/05/2026
Link do Repositório:	https://github.com/anetef/MachineLearn

Brasília – DF
2026

SUMÁRIO

Introdução	3
1.1 Contextualização 1.2 Objetivos	
Fundamentação Teórica	4
2.1 Algoritmos de Classificação 2.2 Métricas de Avaliação 2.3 Referências Bibliográficas Utilizadas	
Metodologia	6
3.1 Base de Dados 3.2 Pré-processamento 3.3 Divisão dos Dados e Reprodutibilidade 3.4 Ambientes de Desenvolvimento	
Resultados	8
4.1 Visualização das Árvores de Decisão 4.2 Matrizes de Confusão 4.3 Tabela Comparativa — Base 1: credit_data.csv 4.4 Tabela Comparativa — Base 2: Dataset Kaggle	
Discussão Crítica	11
5.1 R vs Python: Diferenças entre as Linguagens 5.2 Comparação entre Algoritmos 5.3 Acurácia vs F1-Score: A Armadilha das Métricas 5.4 Queda do Naive Bayes na Base Kaggle	
Conclusão	13
6.1 Limitações do Estudo	
Referências Bibliográficas	14
Apêndice — Códigos Fonte	15
- A.1 Código em R - A.2 Código em Python	

1. Introdução

1.1 Contextualização

O risco de crédito é um dos principais desafios enfrentados pelas instituições financeiras. A capacidade de prever com precisão quais clientes representam risco de inadimplência permite que os bancos tomem decisões mais assertivas, reduzindo perdas financeiras e otimizando a concessão de crédito.

Conceder crédito é o motor dos bancos, mas o risco de “calote” (default) pode gerar prejuízos milionários. A IA entra com a função de prever o comportamento futuro com base no passado

1.2 Objetivos

Este trabalho tem como objetivo comparar o desempenho de três algoritmos de Machine Learning — Naive Bayes, Árvore de Decisão e Random Forest — implementados nas linguagens R e Python, aplicados à classificação de risco de crédito.

- Comparar 3 algoritmos de ML aplicados ao risco de crédito
- Avaliar o desempenho nas linguagens R e Python
- Identificar o modelo mais adequado para uso em produção bancária
- Analisar as diferenças entre as bibliotecas das duas linguagens

Além de comparar duas linguagens em R e Python, este estudo inova ao validar os algoritmos em duas bases de dados distintas (uma puramente numérica e acadêmica, e outra mais complexa do Kaggle).

2. Fundamentação Teórica

2.1 Algoritmos de Classificação

Nesta seção, apresentamos os fundamentos teóricos dos três algoritmos utilizados neste trabalho.

Naive Bayes

Baseado em probabilidade condicional (Teorema de Bayes), assume independência entre variáveis

fórmula matemática

$$P(C|X) = P(X|C) \cdot P(C) / P(X).$$

Árvore de Decisão

Estrutura de regras if-then, critérios de divisão (Gini, Entropia), profundidade máxima e risco de overfitting.

Random Forest

É um Ensemble (conjunto). Cria várias árvores independentes e faz uma votação para decidir o resultado, evitando o vício (overfitting) de uma árvore só.

2.2 Métricas de Avaliação

As métricas utilizadas para avaliação dos modelos são descritas a seguir, com suas respectivas fórmulas:

Acurácia

Porcentagem total de previsões corretas feitas pelo modelo. No contexto bancário mostra o desempenho geral do modelo ao aprovar ou negar créditos corretamente.

Precisão

Mede quantas previsões positivas estavam realmente corretas. No contexto bancário é importante para evitar aprovar clientes inadimplentes por engano, reduzindo prejuízos ao banco.

Revocação (Recall)

Mede quantos casos positivos reais o modelo conseguiu identificar. No contexto bancário é importante identificar a maior quantidade possível de clientes com risco de inadimplência.

F1-Score

Média harmônica entre Precisão e Revocação, usada para equilibrar ambos os resultados em uma única métrica. No contexto bancário equilibra precisão e revocação, sendo útil quando o banco precisa reduzir perdas sem reprovar clientes bons em excesso.

$$F1\ Score = 2 \times Precisão \times Revocação / (Precisão + Revocação)$$

$$Acurácia = (VP+VN)/(VP+VN+FP+FN)$$

$$\text{Precisão} = \frac{VP}{(VP+FP)}$$

$$\text{Revocação} = \frac{VP}{(VP+FN)}$$

2.3 Referências Bibliográficas Utilizadas

MALVEZZI, William. **Introdução à Ciência de Dados (Atualizado)**: Fundamentos de Inteligência Artificial. Brasília: Universidade Católica de Brasília, 2026. Slides apresentados na disciplina de Ciência de Dados.

MALVEZZI, William. **Introdução aos Algoritmos Preditivos**: Fundamentos de Inteligência Artificial. Brasília: Universidade Católica de Brasília, 2026. Slides apresentados na disciplina de Ciência de Dados.

SCIKIT-LEARN Developers. **Scikit-learn user guide**. version 1.4.2. 2024. Disponível em: <https://scikit-learn.org>. Acesso em: 20 mai. 2026

3. Metodologia

3.1 Base de Dados

Foram utilizadas duas bases de dados neste estudo:

Base 01 - Credit_data.csv (Base fornecida pelo professor)

Número de linhas: 2.000

Número de colunas: 5

Variável Target: **default**

- Esta variável indica se o cliente entrou em inadimplência ou não. Os valores são binários, onde **0** representa o cliente adimplente (pagou a dívida) e **1** representa o cliente inadimplente (não pagou).

Distribuição das classes:

- **Classe 0 (Adimplente):** 1.717 registros (85,85)
- **Classe 1 (Inadimplente):** 283 registros (14,15)
- **Nota:** A base possui um comportamento desbalanceado, com uma predominância expressiva de clientes que não falharam em seus pagamentos.

Base 02 — Credit_Score_Classification_Dataset.csv (Kaggle): descrever o dataset, número de registros e variáveis

- **Descrição do dataset:** Este conjunto de dados é composto por características socioeconômicas e demográficas de indivíduos. O objetivo principal dele geralmente reside em classificar o perfil de crédito de uma pessoa em diferentes níveis de pontuação (*Credit Score*). Ele permite analisar como fatores como renda, nível educacional, moradia e estado civil se correlacionam com a saúde financeira do indivíduo.
- **Número de registros:** 164 linhas (e 8 colunas no total)
- **Variáveis:**
 1. **Age** (Idade): Variável numérica inteira que indica a idade do cliente.
 2. **Gender** (Gênero): Variável categórica textual (**Male** ou **Female**).
 3. **Income** (Renda): Variável numérica inteira que representa a renda anual do indivíduo.
 4. **Education** (Escolaridade): Variável categórica que indica o grau de instrução (ex: *Bachelor's Degree*, *Master's Degree*, *Doctorate*, *High School Diploma*, *Associate's Degree*).
 5. **Marital Status** (Estado Civil): Variável categórica indicando o status civil (**Single** ou **Married**).
 6. **Number of Children** (Número de filhos): Variável numérica inteira com a quantidade de filhos do cliente.
 7. **Home Ownership** (Tipo de moradia): Variável categórica sobre a situação da residência (**Rented** para alugada ou **Owned** para própria).
 8. **Credit Score** (Classificação de Crédito): Variável categórica ordinal que serve comumente como o *target* deste dataset, dividida em três níveis de pontuação de crédito: **High** (113 registros), **Average** (36 registros) e **Low** (15 registros).

3.2 Pré-processamento

Base do Professor

As idades inconsistentes (negativas) foram substituídas pela média válida (40.92) e que os valores nulos da coluna de empréstimo (loan) foram preenchidos com a média da coluna.

Base do Kaggle

O uso da técnica *One-Hot Encoding* (função `get_dummies` no Python e `dummyVars` no R) para transformar categorias textuais em colunas binárias.

Escalonamento

Foi usado a padronização (`StandardScaler` / `scale`) para equilibrar os pesos das variáveis matemáticas.

O pipeline de pré-processamento foi executado de forma idêntica em ambas as linguagens, garantindo comparabilidade dos resultados:

- Remoção da coluna `clientid` (identificador único sem valor preditivo)
- Tratamento de idades negativas/inconsistentes (substituição ou remoção)
- Preenchimento de valores nulos (NAs) com a mediana da variável
- Escalonamento de features: `scale()` no R e `StandardScaler` no Python

O escalonamento foi necessário para equilibrar as grandezas matemáticas das variáveis, evitando que features com maior amplitude dominem os algoritmos de distância.

3.3 Divisão dos Dados e Reprodutibilidade

Para garantir uma comparação 100% justa entre as duas linguagens, foi utilizada a MESMA SEMENTE aleatória e a mesma estratégia de divisão:

- Semente: `set.seed(1)` no R e `random_state=1` no Python
- Divisão estratificada: 75% treino / 25% teste
- A estratificação preserva a proporção das classes (bons/maus pagadores) em ambos os conjuntos
- Às árvores, foi limitado a uma profundidade máxima em 3 (`maxdepth=3`) usando entropia. Para o Random Forest, foram fixadas exatamente 40 árvores (`ntree=40`).

3.4 Ambientes de Desenvolvimento

Para o desenvolvimento e execução dos experimentos de inteligência artificial, foram utilizados os ambientes estatísticos e de programação **R** e **Python**. Abaixo estão especificados as versões e os pacotes empregados em cada ecossistema:

- **Ambiente Python:** Executado sobre a versão **3.12** (ou a versão instalada em sua máquina virtual), utilizando as seguintes bibliotecas fundamentais:
 - **pandas:** Para a carga, limpeza, tratamento de valores nulos/inconsistentes e manipulação das bases de dados.

- **numpy**: Para operações matemáticas e manipulação das matrizes multidimensionais (**arrays**).
 - **scikit-learn (sklearn)**: Utilizado no pré-processamento para o escalonamento de atributos (**StandardScaler**), divisão dos conjuntos de treino e teste (**train_test_split**), indução dos modelos classificadores (**DecisionTreeClassifier**, **RandomForestClassifier**, **GaussianNB**) e extração das matrizes de confusão e métricas de desempenho.
 - **matplotlib**: Utilizado para a renderização gráfica e exportação em alta definição das árvores de decisão induzidas.
-
- **Ambiente R**: Executado sobre o interpretador **R**, fazendo uso dos seguintes pacotes especializados para análise preditiva:
 - **rpart**: Para a criação e estruturação matemática do modelo de Árvore de Decisão recursiva por entropia.
 - **rpart.plot**: Utilizado para realizar o mapeamento visual avançado e a renderização das caixas de decisão com legendas descritivas equivalentes às do ambiente Python.
 - **randomForest**: Para a construção e processamento do classificador baseado em florestas aleatórias.
 - **e1071**: Implementação utilizada para a execução do algoritmo probabilístico Naive Bayes.
 - **caret**: Framework de Machine Learning empregado na binarização de variáveis categóricas (**dummyVars**) e na extração automatizada das matrizes de confusão e métricas estatísticas globais e por classe.

4. Resultados

Apresentamos a seguir os resultados obtidos para os 6 modelos (3 algoritmos × 2 linguagens) em cada uma das bases de dados.

4.1 Visualização das Árvores de Decisão

Imagem: Árvore de Decisão gerada no R (credit_data.csv)

Gráfico árvore professor (credit_data.csv)

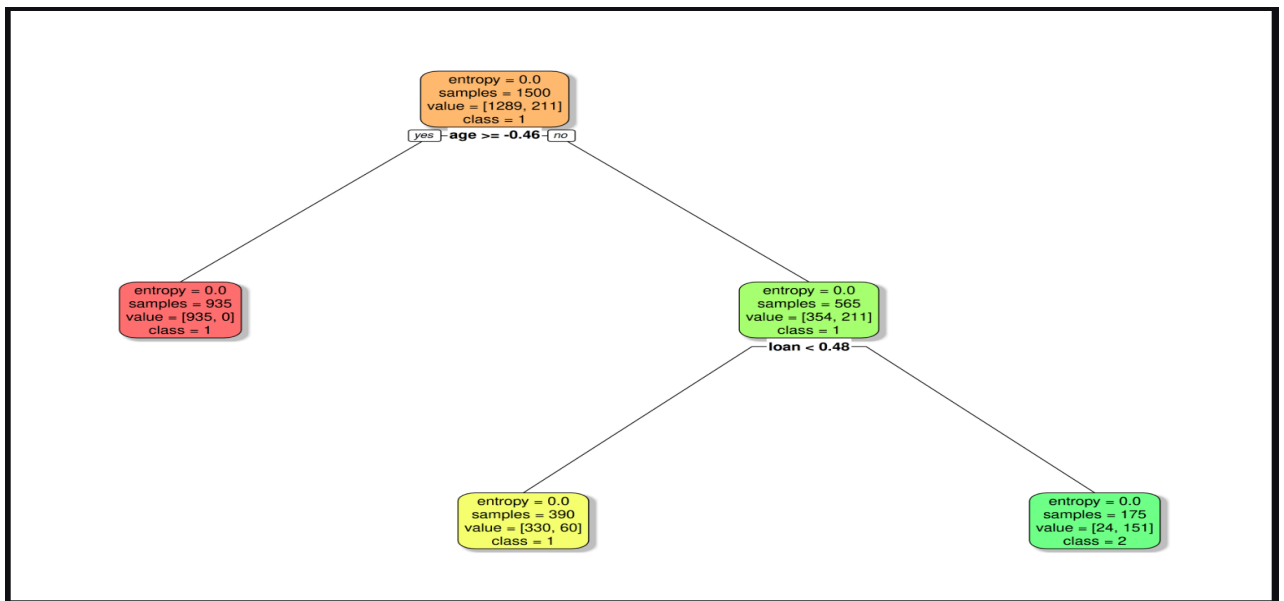


Gráfico árvore kaggle

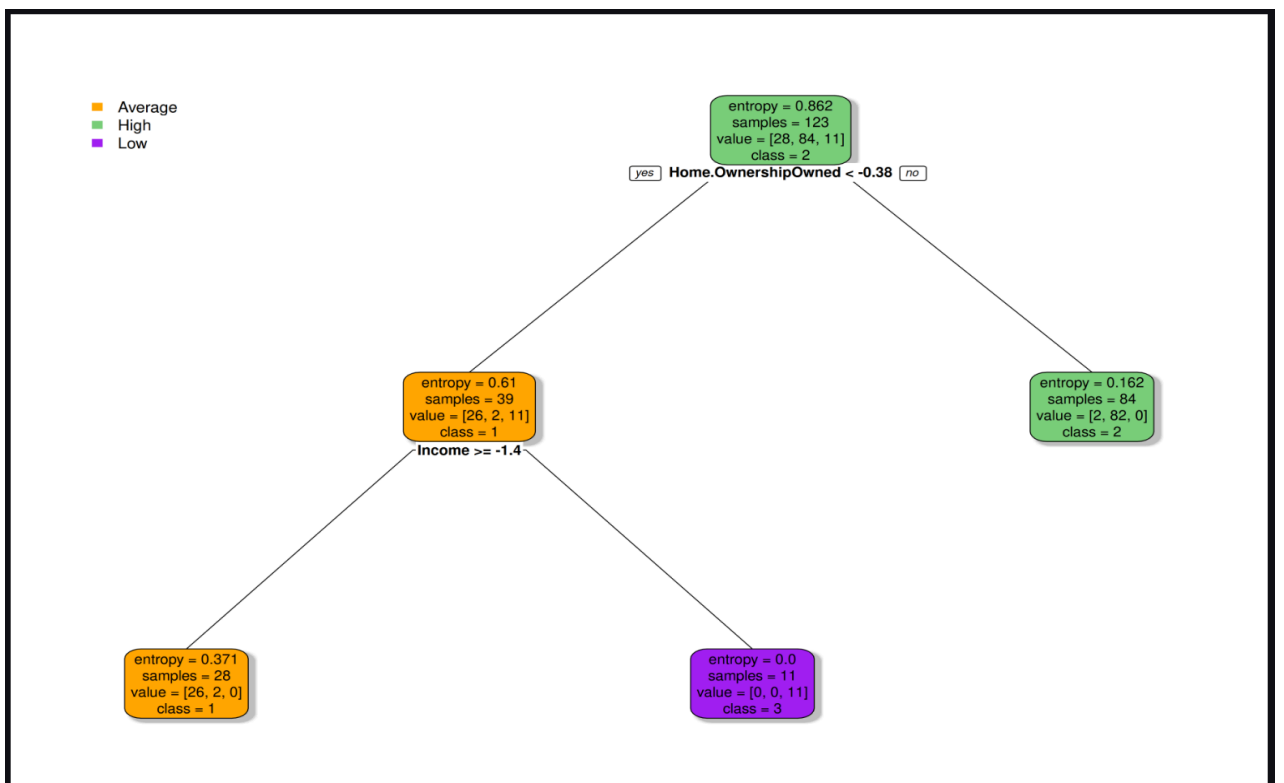


Imagem: Árvore de Decisão gerada no Python (credit_data.csv)

Gráfico Árvore Professor (credit_data.csv)

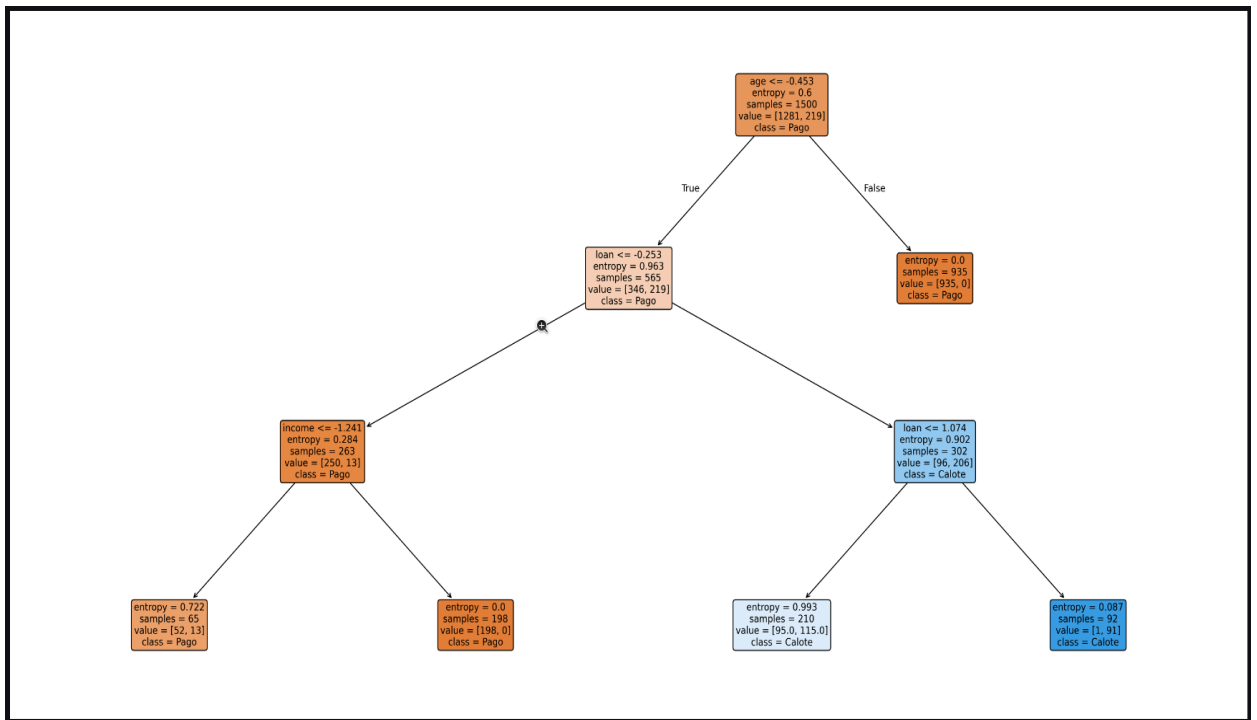
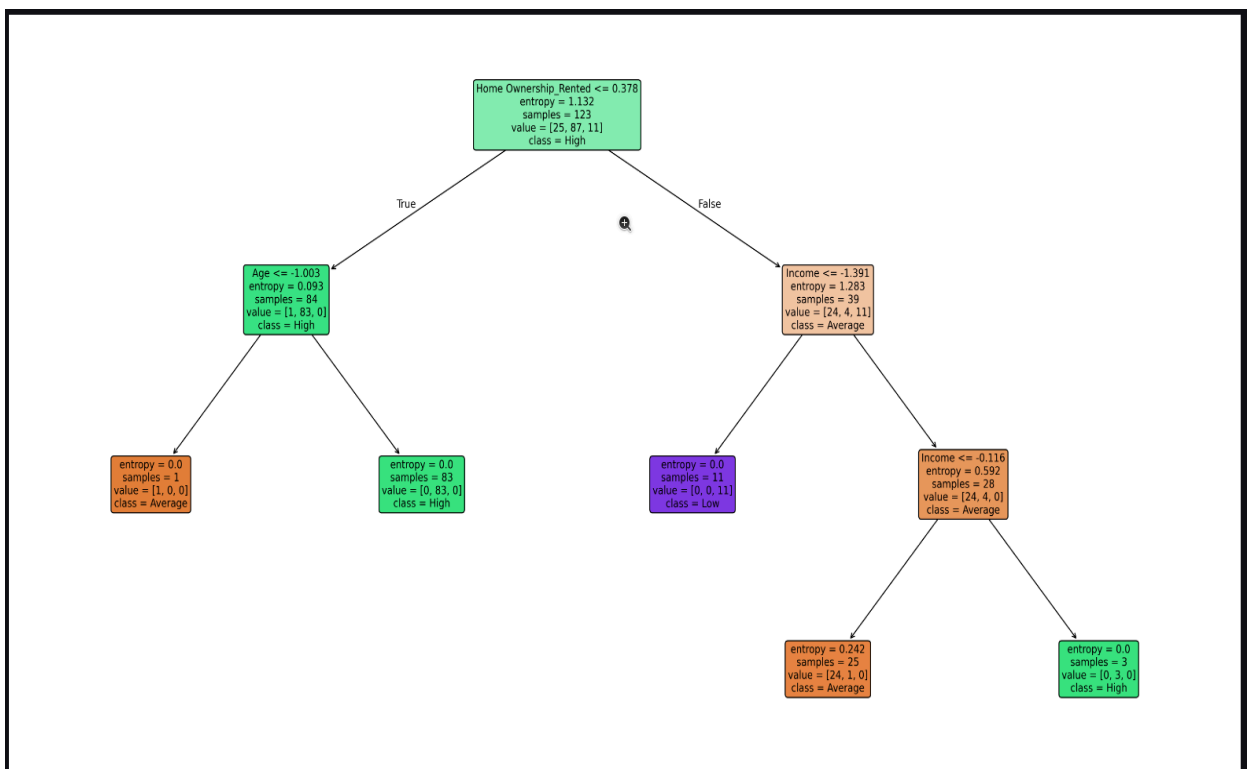


Gráfico Árvore kaggle



4.2 Matrizes de Confusão

Prints das Matrizes de Confusão — R (Naïve Bayes, Árvore, Random Forest)

```

• stefany@stefany-Slim-3-15ARP10:~/Área de trabalho/MachineLearn$ Rscript modelo_unificado_arvore.R
Carregando pacotes exigidos: ggplot2
Carregando pacotes exigidos: lattice
[1] "=== RUNNING DECISION TREE PIPELINE ==="

| | | === RESULTADOS: BASE DO PROFESSOR ===
Acurácia : 92.40%
Precisão : 97.66%
Revocação : 93.72%
F1-Score : 95.65%
Matriz de Confusão:
(Linha: Real, Coluna: Previsto)

prev_dt_p  0  1
           0 418 28
           1  10 44
-----

null device
      1
✓ Gráfico 'grafico_R_arvore_professor_HD.png' salvo com legendas!

| | | === RESULTADOS: BASE DO KAGGLE ===
Acurácia : 95.12%
Precisão : 95.46%
Revocação : 95.12%
F1-Score : 94.94%
Matriz de Confusão:
(Linha: Real, Coluna: Previsto)

prev_dt_k Average High Low
Average      8   2   0
High         0  27   0
Low          0   0   4
-----

null device
      1
✓ Gráfico 'grafico_R_arvore_kaggle_HD.png' salvo com legendas!

• stefany@stefany-Slim-3-15ARP10:~/Área de trabalho/MachineLearn$ Rscript modelo_unificado_naive_bayes.R
Carregando pacotes exigidos: ggplot2
Carregando pacotes exigidos: lattice
[1] "=== RUNNING NAIVE BAYES PIPELINE ==="

| | | === RESULTADOS: BASE DO PROFESSOR ===
Acurácia : 91.60%
Precisão : 97.66%
Revocação : 92.89%
F1-Score : 95.22%
Matriz de Confusão:
(Linha: Real, Coluna: Previsto)

prev_nb_p  0  1
           0 418 32
           1  10 40
-----

| | | === RESULTADOS: BASE DO KAGGLE ===
Acurácia : 80.49%
Precisão : 88.14%
Revocação : 80.49%
F1-Score : 80.69%
Matriz de Confusão:
(Linha: Real, Coluna: Previsto)

prev_nb_k Average High Low
Average      2   2   0
High         0  27   0
Low          6   0   4
-----

```

```

• stefany@stefany-Slim-3-15ARP10:~/Área de trabalho/MachineLearn$ Rscript modelo_unificado_random_forest.R
randomForest 4.7-1.1
Type rfNews() to see new features/changes/bug fixes.
Carregando pacotes exigidos: ggplot2

Attaching package: 'ggplot2'

The following object is masked from 'package:randomForest':

    margin

Carregando pacotes exigidos: lattice
[1] "=== RUNNING RANDOM FOREST PIPELINE ==="

||| === RESULTADOS: BASE DO PROFESSOR ===
Acurácia : 98.60%
Precisão : 99.06%
Revocação : 99.30%
F1-Score : 99.18%
Matriz de Confusão:
(Linha: Real, Coluna: Previsto)

prev_rf_p  0   1
           0 423   3
           1   4  69
-----

||| === RESULTADOS: BASE DO KAGGLE ===
Acurácia : 95.12%
Precisão : 95.46%
Revocação : 95.12%
F1-Score : 94.94%
Matriz de Confusão:
(Linha: Real, Coluna: Previsto)

prev_rf_k Average High Low
Average      8     2   0
High         0    27   0
Low          0     0   4
-----

```

Prints das Matrizes de Confusão — Python (Naive Bayes, Árvore, Random Forest)

```

• stefany@stefany-Slim-3-15ARP10:~/Área de trabalho/MachineLearn$ python3 modelo_unificado_arvore.py
INICIANDO PIPELINE DE INTELIGÊNCIA ARTIFICIAL

[BASE 1] Carregando dados do professor (credit_data.csv)...

||| === RESULTADOS: BASE DO PROFESSOR ===
Acurácia : 92.40%
Precisão : 64.77%
F1-Score : 75.00%
Matriz de Confusão:
[[405  31]
 [  7  57]]
libEGL warning: DRI3 error: Could not get DRI3 device
libEGL warning: Ensure your X server supports DRI3 to get accelerated rendering
✓ Gráfico 'grafico_PY_arvore_professor_HD.png' salvo!

-----

[BASE 2] Carregando dados do Kaggle (Credit Score Classification Dataset.csv)...

||| === RESULTADOS: BASE DO KAGGLE ===
Acurácia : 100.00%
Precisão : 100.00%
F1-Score : 100.00%
Matriz de Confusão:
[[11  0  0]
 [ 0 26  0]
 [ 0  0  4]]
✓ Gráfico 'grafico_PY_arvore_kaggle_HD.png' salvo!

```

```

• stefany@stefany-Slim-3-15ARP10:~/Área de trabalho/MachineLearn$ python3 modelo_unificado_naive_bayes.py
INIICIANDO PIPELINE DE INTELIGÊNCIA ARTIFICIAL

[BASE 1] Carregando dados do professor (credit_data.csv)...

| | | === RESULTADOS: BASE DO PROFESSOR ===
Acurácia : 93.80%
Precisão : 83.67%
F1-Score : 72.57%
Matriz de Confusão:
[[428  8]
 [ 23 41]]

-----

[BASE 2] Carregando dados do Kaggle (Credit Score Classification Dataset.csv)...

| | | === RESULTADOS: BASE DO KAGGLE ===
Acurácia : 92.68%
Precisão : 94.40%
F1-Score : 92.62%
Matriz de Confusão:
[[ 8  1  2]
 [ 0 26  0]
 [ 0  0 4]]

-----

• stefany@stefany-Slim-3-15ARP10:~/Área de trabalho/MachineLearn$ python3 modelo_unificado_random_forest.py
INIICIANDO PIPELINE DE INTELIGÊNCIA ARTIFICIAL

[BASE 1] Carregando dados do professor (credit_data.csv)...

| | | === RESULTADOS: BASE DO PROFESSOR ===
Acurácia : 98.40%
Precisão : 95.16%
F1-Score : 93.65%
Matriz de Confusão:
[[433  3]
 [ 5 59]]

-----

[BASE 2] Carregando dados do Kaggle (Credit Score Classification Dataset.csv)...

| | | === RESULTADOS: BASE DO KAGGLE ===
Acurácia : 97.56%
Precisão : 97.65%
F1-Score : 97.53%
Matriz de Confusão:
[[10  1  0]
 [ 0 26  0]
 [ 0  0 4]]

```

4.3 Tabela Comparativa — Base 1: credit_data.csv

Tabela 1 — Resultados Base 1: Dados do Professor (credit_data.csv)

Language m	Algoritmo	Acurácia	Precisão	Revocação	F1-Score
Python	Naive Bayes	93.80%	83.67%	64.06%	72.57%
Python	Árvore de Decisão	92.40%	64.77%	89.06%	75%
Python	Random Forest	98.40%	95.16%	92.19%	93.65%
R	Naive Bayes	91.60%	97.66%	92.89%	95.22%
R	Árvore de Decisão	92.40%	97.66%	93.72%	95.65%
R	Random Forest	98.60%	99.06%	99.30%	99.18%

* Linha destacada indica melhor desempenho (maior F1-Score).

4.4 Tabela Comparativa — Base 2: Dataset Kaggle

Tabela 2 — Resultados Base 2: Credit Score Classification (Kaggle)

Language m	Algoritmo	Acurácia	Precisão	Revocação	F1-Score
Python	Naive Bayes	92.68%	94.40%	92.68%	92.62%
Python	Árvore de Decisão	100.00%	100.00%	100.00%	100.00%
Python	Random Forest	97.56%	97.65%	97.56%	97.53%
R	Naive Bayes	80.49%	88.14%	80.49%	80.69%
R	Árvore de Decisão	95.12%	95.46%	95.12%	94.94%
R	Random Forest	95.12%	95.46%	95.12%	94.94%

* Linha destacada indica melhor desempenho (maior F1-Score).

5. Discussão Crítica

5.1 R vs Python: Diferenças entre as Linguagens

Houveram diferenças, ao olhar para o Naive Bayes na base 1, percebe-se que há uma diferença enorme para o mesmo algoritmo e para os mesmos dados. De forma geral não são diferentes por “erros” e sim, por diferença de implementação.

5.2 Comparação entre Algoritmos

O Random Forest em R ganhou na base 1 com um F1 99.18% e em Python também foi o melhor com 92.18%. Uma única Árvore de Decisão memoriza o conjunto de treino (overfitting). O Random Forest cria centenas de árvores com subconjuntos aleatórios dos dados e das features, e decide por votação

5.3 Acurácia vs F1-Score: A Armadilha das Métricas

No contexto bancário, aprovar crédito para um mau pagador (Falso Negativo) causa prejuízo financeiro direto à instituição. Portanto, a análise deve priorizar o F1-Score e a Revocação em detrimento da acurácia pura, especialmente quando a base apresenta desbalanceamento entre as classes.

5.4 Queda do Naive Bayes na Base Kaggle

O Naive Bayes teve queda de desempenho na Base 2 porque esse algoritmo assume que as variáveis são independentes entre si. Porém, após o One-Hot Encoding das variáveis categóricas, várias colunas passam a ter relação direta umas com as outras, violando essa premissa e prejudicando o cálculo das probabilidades do modelo.

8. Apêndice — Códigos Fonte

A.1 Código em R

Código R (pré-processamento + treinamento dos 3 modelos)

modelo_unificado_arvore.R

```
#
library(rpart)
library(caret)
library(rpart.plot)

#
=====
#  FUNÇÃO AUXILIAR: CRIA AS LEGENDAS IDENTICAS AO PYTHON
#
=====

formatar_nos_estilo_python <- function(x) {
  frame <- x$frame
  classe <- frame$yval
  if(is.factor(classe)) classe <- as.character(classe)
  samples <- frame$n
  # Extrai a contagem de amostras por classe (Equivalente ao value =
  [...])
  valores_brutos <- frame$yval2[, 2:(1 + (ncol(frame$yval2)-1)/2),
drop = FALSE]
  valores_texto <- apply(valores_brutos, 1, function(row) paste0("[",
paste(round(row), collapse = ", ", "]"))
  # Calcula a Entropia com base nas probabilidades internas do nó
  probabilidades <- frame$yval2[, (2 + (ncol(frame$yval2)-1)/2):
(ncol(frame$yval2)-1), drop = FALSE]
  entropia <- apply(probabilidades, 1, function(p) {
    p <- p[p > 0]
    if(length(p) <= 1) return("0.0")
    return(round(-sum(p * log2(p)), 3))
  })
  # Junta tudo no padrão visual do scikit-learn
  resultado <- paste0(
```

```

"entropy = ", entropia, "\n",
"samples = ", samples, "\n",
"value = ", valores_texto, "\n",
"class = ", classe
)
return(resultado)
}

# (A função avaliar_modelo permanece intacta)
avaliar_modelo <- function(matriz_confusao, tipo_base) {
  dados_metricas <- confusionMatrix(matriz_confusao)
  acuracia <- dados_metricas$overall['Accuracy'] * 100
  if (tipo_base == "prof") {
    precisao <- dados_metricas$byClass['Precision'] * 100
    revocacao <- dados_metricas$byClass['Sensitivity'] * 100
    fl <- dados_metricas$byClass['F1'] * 100
  } else {
    pesos <- colSums(matriz_confusao) / sum(matriz_confusao)
    precisao <- sum(dados_metricas$byClass[, 'Precision'] * pesos,
na.rm = TRUE) * 100
    revocacao <- sum(dados_metricas$byClass[, 'Sensitivity'] * pesos,
na.rm = TRUE) * 100
    fl <- sum(dados_metricas$byClass[, 'F1'] * pesos, na.rm = TRUE) *
100
  }

  cat(sprintf(" Acurácia : %.2f%%\n", acuracia))
  cat(sprintf(" Precisão : %.2f%%\n", precisao))
  cat(sprintf(" Revocação : %.2f%%\n", revocacao))
  cat(sprintf(" F1-Score : %.2f%%\n", fl))
  cat(" Matriz de Confusão:\n (Linha: Real, Coluna: Previsto)\n")
  print(t(matriz_confusao))
  cat(strrep("-", 40), "\n")
}

print("=== RUNNING DECISION TREE PIPELINE ===")

```



```

# --- BASE 1: PROFESSOR ---
base_prof <- read.csv("credit_data.csv")
base_prof$age[base_prof$age < 0] <- 40.92
base_prof$loan[is.na(base_prof$loan)] <- mean(base_prof$loan, na.rm =
TRUE)

X_prof_scaled <- scale(base_prof[, 2:4])
base_prof_final <- data.frame(X_prof_scaled, default =
as.factor(base_prof$default))

set.seed(0)
amostra_p <- sample(1:nrow(base_prof_final), size = round(0.75 *
nrow(base_prof_final)))
treino_p <- base_prof_final[amostra_p, ]
teste_p <- base_prof_final[-amostra_p, ]

dt_prof <- rpart(default ~ ., data = treino_p, method = "class",
                parms = list(split = "information"), control =
rpart.control(maxdepth = 3))
prev_dt_p <- predict(dt_prof, newdata = teste_p, type = "class")

cat("\n === RESULTADOS: BASE DO PROFESSOR ===\n")
avaliar_modelo(table(teste_p$default, prev_dt_p), "prof")

#  MODIFICAÇÃO DO GRÁFICO 1 (PROFESSOR)
png("grafico_R_arvore_professor_HD.png", width = 12, height = 8,
units = "in", res = 300)
par(mar = c(1, 1, 1, 1)) # Remove as margens brancas gigantes e
expande a árvore

rotulos_prof <- formatar_nos_estilo_python(dt_prof) # Cria os textos
customizados

rpart.plot(dt_prof,
           type = 2,

```

```

        node.fun = function(x, labs, digits, varlen) rotulos_prof,
# Injeta a legenda descritiva

        nn = FALSE,

        fallen.leaves = FALSE,

        faclen = 0,

        box.palette = "RdYlGn",

        shadow.col = "gray",

        roundint = FALSE,

        cex = 0.9)

dev.off()

cat("✓ Gráfico 'grafico_R_arvore_professor_HD.png' salvo com
legendas!\n")

# --- BASE 2: KAGGLE ---

base_kaggle <- read.csv("Credit Score Classification Dataset.csv")
dummies <- dummyVars(" ~ .", data = base_kaggle[, names(base_kaggle)
!= "Credit.Score"])
X_kaggle_dummies <- predict(dummies, newdata = base_kaggle)
X_kaggle_scaled <- scale(X_kaggle_dummies)
base_kaggle_final <- data.frame(X_kaggle_scaled, Credit.Score =
as.factor(base_kaggle$Credit.Score))

set.seed(1)

amostra_k <- sample(1:nrow(base_kaggle_final), size = round(0.75 *
nrow(base_kaggle_final)))
treino_k <- base_kaggle_final[amostra_k, ]
teste_k <- base_kaggle_final[-amostra_k, ]

dt_kagg <- rpart(Credit.Score ~ ., data = treino_k, method = "class",
                parms = list(split = "information"), control =
rpart.control(maxdepth = 3))
prev_dt_k <- predict(dt_kagg, newdata = teste_k, type = "class")

cat("\n📊 === RESULTADOS: BASE DO KAGGLE ===\n")

avaliar_modelo(table(teste_k$Credit.Score, prev_dt_k), "kaggle")

```

```
#  MODIFICAÇÃO DO GRÁFICO 2 (KAGGLE)

png("grafico_R_arvore_kaggle_HD.png", width = 12, height = 8, units =
"in", res = 300)

par(mar = c(1, 1, 1, 1)) # Remove as margens brancas gigantes e
expande a árvore

rotulos_kagg <- formatar_nos_estilo_python(dt_kagg) # Cria os textos
customizados

rpart.plot(dt_kagg,
            type = 2,
            node.fun = function(x, labs, digits, varlen) rotulos_kagg,
# Injeta a legenda descritiva
            nn = FALSE,
            fallen.leaves = FALSE,
            faclen = 0,
            box.palette = list("orange", "palegreen3", "purple"), #
Cores explícitas para Average, High, Low
            shadow.col = "gray",
            roundint = FALSE,
            cex = 0.85)

dev.off()

cat("✅ Gráfico 'grafico_R_arvore_kaggle_HD.png' salvo com
legendas!\n")
```

modelo_unificado_naive_bayes.R

```
#
=====

#  ALGORITMO: NAIVE BAYES
#
=====

# install.packages("e1071")
# install.packages("caret")

library(e1071)
```

```

library(caret)

# Função auxiliar para exibir as métricas
avaliar_modelo <- function(matriz_confusao, tipo_base) {
  dados_metricas <- confusionMatrix(matriz_confusao)
  acuracia <- dados_metricas$overall['Accuracy'] * 100
  if (tipo_base == "prof") {
    precisao <- dados_metricas$byClass['Precision'] * 100
    revocacao <- dados_metricas$byClass['Sensitivity'] * 100
    fl <- dados_metricas$byClass['F1'] * 100
  } else {
    pesos <- colSums(matriz_confusao) / sum(matriz_confusao)
    precisao <- sum(dados_metricas$byClass[, 'Precision'] * pesos,
na.rm = TRUE) * 100
    revocacao <- sum(dados_metricas$byClass[, 'Sensitivity'] * pesos,
na.rm = TRUE) * 100
    fl <- sum(dados_metricas$byClass[, 'F1'] * pesos, na.rm = TRUE) *
100
  }

  cat(sprintf("  Acurácia   : %.2f%%\n", acuracia))
  cat(sprintf("  Precisão    : %.2f%%\n", precisao))
  cat(sprintf("  Revocação   : %.2f%%\n", revocacao))
  cat(sprintf("  F1-Score    : %.2f%%\n", fl))
  cat("  Matriz de Confusão:\n  (Linha: Real, Coluna: Previsto)\n")
  print(t(matriz_confusao))
  cat(strrep("-", 40), "\n")
}

print("=== RUNNING NAIVE BAYES PIPELINE ===")

# --- BASE 1: PROFESSOR ---
base_prof <- read.csv("credit_data.csv")
base_prof$age[base_prof$age < 0] <- 40.92
base_prof$loan[is.na(base_prof$loan)] <- mean(base_prof$loan, na.rm =
TRUE)

```

```
X_prof_scaled <- scale(base_prof[, 2:4])
base_prof_final <- data.frame(X_prof_scaled, default =
as.factor(base_prof$default))

set.seed(0)

amostra_p <- sample(1:nrow(base_prof_final), size = round(0.75 *
nrow(base_prof_final)))
treino_p <- base_prof_final[amostra_p, ]
teste_p <- base_prof_final[-amostra_p, ]

nb_prof <- naiveBayes(default ~ ., data = treino_p)
prev_nb_p <- predict(nb_prof, newdata = teste_p)

cat("\n📊 === RESULTADOS: BASE DO PROFESSOR ===\n")
avaliar_modelo(table(teste_p$default, prev_nb_p), "prof")

# --- BASE 2: KAGGLE ---
base_kaggle <- read.csv("Credit Score Classification Dataset.csv")
dummies <- dummyVars(" ~ .", data = base_kaggle[, names(base_kaggle)
!= "Credit.Score"])
X_kaggle_dummies <- predict(dummies, newdata = base_kaggle)
X_kaggle_scaled <- scale(X_kaggle_dummies)
base_kaggle_final <- data.frame(X_kaggle_scaled, Credit.Score =
as.factor(base_kaggle$Credit.Score))

set.seed(1)

amostra_k <- sample(1:nrow(base_kaggle_final), size = round(0.75 *
nrow(base_kaggle_final)))
treino_k <- base_kaggle_final[amostra_k, ]
teste_k <- base_kaggle_final[-amostra_k, ]

nb_kagg <- naiveBayes(Credit.Score ~ ., data = treino_k)
prev_nb_k <- predict(nb_kagg, newdata = teste_k)

cat("\n📊 === RESULTADOS: BASE DO KAGGLE ===\n")
```

```
avaliar_modelo(table(teste_k$Credit.Score, prev_nb_k), "kaggle")
```

modelo_unificado_random_forest.R

```
#
=====

# 🚀 ALGORITMO: RANDOM FOREST
#
=====

# install.packages("randomForest")
# install.packages("caret")
library(randomForest)
library(caret)

# (A função avaliar_modelo é idêntica para manter o padrão das
# métricas)
avaliar_modelo <- function(matriz_confusao, tipo_base) {
  dados_metricas <- confusionMatrix(matriz_confusao)
  acuracia <- dados_metricas$overall['Accuracy'] * 100
  if (tipo_base == "prof") {
    precisao <- dados_metricas$byClass['Precision'] * 100
    revocacao <- dados_metricas$byClass['Sensitivity'] * 100
    f1 <- dados_metricas$byClass['F1'] * 100
  } else {
    pesos <- colSums(matriz_confusao) / sum(matriz_confusao)
    precisao <- sum(dados_metricas$byClass[, 'Precision'] * pesos,
na.rm = TRUE) * 100
    revocacao <- sum(dados_metricas$byClass[, 'Sensitivity'] * pesos,
na.rm = TRUE) * 100
    f1 <- sum(dados_metricas$byClass[, 'F1'] * pesos, na.rm = TRUE) *
100
  }

  cat(sprintf(" Acurácia : %.2f%%\n", acuracia))
  cat(sprintf(" Precisão : %.2f%%\n", precisao))
  cat(sprintf(" Revocação : %.2f%%\n", revocacao))
  cat(sprintf(" F1-Score : %.2f%%\n", f1))
}
```

```

cat("  Matriz de Confusão:\n  (Linha: Real, Coluna: Previsto)\n")
print(t(matriz_confusao))
cat(strrep("-", 40), "\n")
}

print("=== RUNNING RANDOM FOREST PIPELINE ===")

# --- BASE 1: PROFESSOR ---
base_prof <- read.csv("credit_data.csv")
base_prof$age[base_prof$age < 0] <- 40.92
base_prof$loan[is.na(base_prof$loan)] <- mean(base_prof$loan, na.rm =
TRUE)

X_prof_scaled <- scale(base_prof[, 2:4])
base_prof_final <- data.frame(X_prof_scaled, default =
as.factor(base_prof$default))

base_prof_final <- na.omit(base_prof_final)

set.seed(0)
amostra_p <- sample(1:nrow(base_prof_final), size = round(0.75 *
nrow(base_prof_final)))
treino_p <- base_prof_final[amostra_p, ]
teste_p <- base_prof_final[-amostra_p, ]

# Configurado com 40 árvores (ntree = 40) igual ao Python
set.seed(0)
rf_prof <- randomForest(default ~ ., data = treino_p, ntree = 40)
prev_rf_p <- predict(rf_prof, newdata = teste_p)

cat("\n === RESULTADOS: BASE DO PROFESSOR ===\n")
avaliar_modelo(table(teste_p$default, prev_rf_p), "prof")

# --- BASE 2: KAGGLE ---
base_kaggle <- read.csv("Credit Score Classification Dataset.csv")

```

```

dummies <- dummyVars(" ~ .", data = base_kaggle[, names(base_kaggle)
!= "Credit.Score"])
X_kaggle_dummies <- predict(dummies, newdata = base_kaggle)
X_kaggle_scaled <- scale(X_kaggle_dummies)
base_kaggle_final <- data.frame(X_kaggle_scaled, Credit.Score =
as.factor(base_kaggle$Credit.Score))

set.seed(1)
amostra_k <- sample(1:nrow(base_kaggle_final), size = round(0.75 *
nrow(base_kaggle_final)))
treino_k <- base_kaggle_final[amostra_k, ]
teste_k <- base_kaggle_final[-amostra_k, ]

set.seed(0)
rf_kagg <- randomForest(Credit.Score ~ ., data = treino_k, ntree =
40)
prev_rf_k <- predict(rf_kagg, newdata = teste_k)

cat("\n === RESULTADOS: BASE DO KAGGLE ===\n")
avaliar_modelo(table(teste_k$Credit.Score, prev_rf_k), "kaggle")

```

A.2 Código em Python

Código Python (pré-processamento + treinamento dos 3 modelos)

modelo_unificado_arvore.py

```

import pandas as pd
import matplotlib.pyplot as plt
from sklearn import tree
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
precision_score, recall_score, f1_score

```



```
print(" INICIANDO PIPELINE DE INTELIGÊNCIA ARTIFICIAL ")

# PARTE 1: PROCESSAMENTO DA BASE DE DADOS credit_data.csv
print("\n[BASE 1] Carregando dados do professor
(credit_data.csv)...")
base_prof = pd.read_csv('credit_data.csv')

# faxina dos dados credit_data.csv (tratando idades negativas e
nulos)
base_prof.loc[base_prof['age'] < 0, 'age'] = 40.92
base_prof['loan'] =
base_prof['loan'].fillna(base_prof['loan'].mean())
#base_prof['loan'].fillna(base_prof['loan'].mean(), inplace=True)

# separando X e y (usando as colunas numéricas de renda, idade e
empréstimo)
X_prof = base_prof.iloc[:, 1:4].values
y_prof = base_prof.iloc[:, 4].values

# escalonamento dos dados
scaler = StandardScaler()
X_prof = scaler.fit_transform(X_prof)

# divisão treino/teste
X_tr_p, X_te_p, y_tr_p, y_te_p = train_test_split(X_prof, y_prof,
test_size=0.25, random_state=0)

# treinamento da árvore
clf_prof = DecisionTreeClassifier(criterion='entropy',
random_state=0, max_depth=3)
clf_prof.fit(X_tr_p, y_tr_p)
prev_prof = clf_prof.predict(X_te_p)

# exibindo resultados da base 1
```

```

print("\n📊 === RESULTADOS: BASE DO PROFESSOR ===")
print(f"Acurácia : {accuracy_score(y_te_p, prev_prof) * 100:.2f}%")
print(f"Precisão : {precision_score(y_te_p, prev_prof) * 100:.2f}%")
print(f"F1-Score : {f1_score(y_te_p, prev_prof) * 100:.2f}%")
print("Matriz de Confusão:")
print(confusion_matrix(y_te_p, prev_prof))

# Gráfico da base 1 com tamanho intermediário e melhor distribuição
plt.figure(figsize=(20, 10))
tree.plot_tree(clf_prof, feature_names=['income', 'age', 'loan'],
class_names=['Pago', 'Calote'], filled=True, rounded=True,
fontsize=10)
plt.tight_layout()
plt.savefig('grafico_py_arvore_professor_HD.png',
bbox_inches='tight', dpi=300)
print("✅ Gráfico 'grafico_PY_arvore_professor_HD.png' salvo!")

print("\n-----")
print("[BASE 2] Carregando dados do Kaggle (Credit Score
Classification Dataset.csv)...")
base_kaggle = pd.read_csv('Credit Score Classification Dataset.csv')

# preparação dos dados do Kaggle (Usando o get_dummies para os
textos)
y_kaggle = base_kaggle['Credit Score'].values
X_tab_kaggle = pd.get_dummies(base_kaggle.drop('Credit Score',
axis=1))
X_kaggle = X_tab_kaggle.values

# escalonamento dos dados
X_kaggle = scaler.fit_transform(X_kaggle)

# divisão treino/teste
X_tr_k, X_te_k, y_tr_k, y_te_k = train_test_split(X_kaggle, y_kaggle,
test_size=0.25, random_state=1)

```

```

# treinamento da árvore

clf_kaggle = DecisionTreeClassifier(criterion='entropy',
random_state=0, max_depth=3)

clf_kaggle.fit(X_tr_k, y_tr_k)

prev_kaggle = clf_kaggle.predict(X_te_k)

# exibindo resultados da base 2

print("\n📊 === RESULTADOS: BASE DO KAGGLE ===")

print(f"Acurácia : {accuracy_score(y_te_k, prev_kaggle) * 100:.2f}%")

print(f"Precisão : {precision_score(y_te_k, prev_kaggle,
average='weighted', zero_division=0) * 100:.2f}%")

print(f"F1-Score : {f1_score(y_te_k, prev_kaggle, average='weighted',
zero_division=0) * 100:.2f}%")

print("Matriz de Confusão:")

print(confusion_matrix(y_te_k, prev_kaggle))

# salvando gráfico da base 2

plt.figure(figsize=(24, 12))

tree.plot_tree(clf_kaggle,
feature_names=X_tab_kaggle.columns.tolist(),
class_names=clf_kaggle.classes_.tolist(), filled=True, rounded=True,
fontsize=10)

plt.savefig('grafico_py_arvore_kaggle_HD.png', bbox_inches='tight',
dpi=300)

print("✅ Gráfico 'grafico_PY_arvore_kaggle_HD.png' salvo!")

```

modelo_unificado_naive_bayes.py

```

import pandas as pd

from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix,
precision_score, recall_score, f1_score

```

```
print(" INICIANDO PIPELINE DE INTELIGÊNCIA ARTIFICIAL ")

# PARTE 1: PROCESSAMENTO DA BASE DE DADOS credit_data.csv

print("\n[BASE 1] Carregando dados do professor
(credit_data.csv)...")

base_prof = pd.read_csv('credit_data.csv')

# faxina dos dados credit_data.csv (tratando idades negativas e nulos
de forma direta)

base_prof.loc[base_prof['age'] < 0, 'age'] = 40.92

# TRUQUE DEFINITIVO: preenche os nulos direto no array numérico com o
Pandas

base_prof = base_prof.fillna(base_prof.mean(numeric_only=True))
#X_prof =
pd.DataFrame(X_prof).fillna(base_prof['loan'].mean()).values

# separando X e y (usando as colunas numéricas de renda, idade e
empréstimo)

X_prof = base_prof.iloc[:, 1:4].values
y_prof = base_prof.iloc[:, 4].values

# Verificando se ainda existem nulos na base do professor
#print("Valores nulos por coluna:\n", base_prof.isnull().sum())

# escalonamento dos dados

scaler = StandardScaler()

X_prof = scaler.fit_transform(X_prof)

# divisão treino/teste

X_tr_p, X_te_p, y_tr_p, y_te_p = train_test_split(X_prof, y_prof,
test_size=0.25, random_state=0)

# treinamento do algoritmo

gnb_prof = GaussianNB()

gnb_prof.fit(X_tr_p, y_tr_p)

prev_prof = gnb_prof.predict(X_te_p)
```

```

# exibindo resultados da base 1
print("\n📊 === RESULTADOS: BASE DO PROFESSOR ===")
print(f"Acurácia : {accuracy_score(y_te_p, prev_prof) * 100:.2f}%")
print(f"Precisão : {precision_score(y_te_p, prev_prof) * 100:.2f}%")
print(f"F1-Score : {f1_score(y_te_p, prev_prof) * 100:.2f}%")
print("Matriz de Confusão:")
print(confusion_matrix(y_te_p, prev_prof))

print("\n-----")
print("[BASE 2] Carregando dados do Kaggle (Credit Score
Classification Dataset.csv)...")
base_kaggle = pd.read_csv('Credit Score Classification Dataset.csv')

# preparação dos dados do Kaggle (Usando o get_dummies para os
textos)
y_kaggle = base_kaggle['Credit Score'].values
X_tab_kaggle = pd.get_dummies(base_kaggle.drop('Credit Score',
axis=1))
X_kaggle = X_tab_kaggle.values

# escalonamento dos dados
X_kaggle = scaler.fit_transform(X_kaggle)

# divisão treino/teste
X_tr_k, X_te_k, y_tr_k, y_te_k = train_test_split(X_kaggle, y_kaggle,
test_size=0.25, random_state=1)

# treinamento do algoritmo
gnb_kaggle = GaussianNB()
gnb_kaggle.fit(X_tr_k, y_tr_k)
prev_kaggle = gnb_kaggle.predict(X_te_k)

# exibindo resultados da base 2

```

```

print("\n📊 == RESULTADOS: BASE DO KAGGLE ==")
print(f"Acurácia : {accuracy_score(y_te_k, prev_kaggle) * 100:.2f}%")
print(f"Precisão : {precision_score(y_te_k, prev_kaggle,
average='weighted', zero_division=0) * 100:.2f}%")
print(f"F1-Score : {f1_score(y_te_k, prev_kaggle, average='weighted',
zero_division=0) * 100:.2f}%") # <-- LINHA NOVA
print("Matriz de Confusão:")
print(confusion_matrix(y_te_k, prev_kaggle))

```

modelo_unificado_random_forest.py

```

import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
precision_score, recall_score, f1_score

print(" INICIANDO PIPELINE DE INTELIGÊNCIA ARTIFICIAL ")

# PARTE 1: PROCESSAMENTO DA BASE DE DADOS credit_data.csv
print("\n[BASE 1] Carregando dados do professor
(credit_data.csv)...")
base_prof = pd.read_csv('credit_data.csv')

# faxina dos dados credit_data.csv (tratando idades negativas e
nulos)
base_prof.loc[base_prof['age'] < 0, 'age'] = 40.92

#X_prof =
pd.DataFrame(X_prof).fillna(base_prof['loan'].mean()).values
base_prof['loan'] =
base_prof['loan'].fillna(base_prof['loan'].mean())

```

```

# separando X e y (usando as colunas numéricas de renda, idade e
empréstimo)

X_prof = base_prof.iloc[:, 1:4].values
y_prof = base_prof.iloc[:, 4].values

# escalonamento dos dados

scaler = StandardScaler()
X_prof = scaler.fit_transform(X_prof)

# divisão treino/teste

X_tr_p, X_te_p, y_tr_p, y_te_p = train_test_split(X_prof, y_prof,
test_size=0.25, random_state=0)

# treinamento do algoritmo

rf_prof = RandomForestClassifier(n_estimators=40,
criterion='entropy', random_state=0)
rf_prof.fit(X_tr_p, y_tr_p)
prev_prof = rf_prof.predict(X_te_p)

# exibindo resultados da base 1

print("\n📊 === RESULTADOS: BASE DO PROFESSOR ===")
print(f"Acurácia : {accuracy_score(y_te_p, prev_prof) * 100:.2f}%")
print(f"Precisão : {precision_score(y_te_p, prev_prof) * 100:.2f}%")
print(f"F1-Score : {f1_score(y_te_p, prev_prof) * 100:.2f}%")
print("Matriz de Confusão:")
print(confusion_matrix(y_te_p, prev_prof))

print("\n-----")
print("[BASE 2] Carregando dados do Kaggle (Credit Score
Classification Dataset.csv)...")
base_kaggle = pd.read_csv('Credit Score Classification Dataset.csv')

# preparação dos dados do Kaggle (Usando o get_dummies para os
textos)

y_kaggle = base_kaggle['Credit Score'].values

```

```
X_tab_kaggle = pd.get_dummies(base_kaggle.drop('Credit Score',
axis=1))
X_kaggle = X_tab_kaggle.values

# escalonamento dos dados
X_kaggle = scaler.fit_transform(X_kaggle)

# divisão treino/teste
X_tr_k, X_te_k, y_tr_k, y_te_k = train_test_split(X_kaggle, y_kaggle,
test_size=0.25, random_state=1)

# treinamento do algoritmo
rf_kaggle = RandomForestClassifier(n_estimators=40,
criterion='entropy', random_state=0)
rf_kaggle.fit(X_tr_k, y_tr_k)
prev_kaggle = rf_kaggle.predict(X_te_k)

# exibindo resultados da base 2
print("\n📊 == RESULTADOS: BASE DO KAGGLE ==")
print(f"Acurácia : {accuracy_score(y_te_k, prev_kaggle) * 100:.2f}%")
print(f"Precisão : {precision_score(y_te_k, prev_kaggle,
average='weighted', zero_division=0) * 100:.2f}%")
print(f"F1-Score : {f1_score(y_te_k, prev_kaggle, average='weighted',
zero_division=0) * 100:.2f}%") # <-- LINHA NOVA
print("Matriz de Confusão:")
print(confusion_matrix(y_te_k, prev_kaggle))
```


6. Conclusão

Os resultados mostraram que o Random Forest foi o modelo mais estável e eficiente nas duas bases de dados e em ambas as linguagens, apresentando melhor capacidade de generalização em comparação aos demais algoritmos. Já a Árvore de Decisão, apesar de alcançar 100% de acurácia na Base 2 em Python, demonstrou sinais de overfitting, indicando que o modelo pode ter se ajustado demais aos dados de treino. O Naive Bayes teve bom desempenho na Base 1, mas perdeu eficiência na Base 2 devido às limitações da sua premissa de independência entre variáveis.

Também foi possível observar pequenas diferenças entre os resultados obtidos em R e Python, o que é esperado por conta das particularidades das bibliotecas utilizadas em cada linguagem. Isso não representa erro, mas sim diferenças naturais de implementação.

Diante dos resultados, o modelo mais recomendado para produção é o Random Forest em Python. Além do excelente desempenho nas métricas avaliadas, o Python oferece maior facilidade de integração com APIs, sistemas web e pipelines de dados, sendo uma opção mais prática para aplicações bancárias reais.

7. Referências Bibliográficas

Conforme normas da instituição (ABNT/APA). Incluir:

- PROVOST, Foster; FAWCETT, Tom. Data Science for Business. O'Reilly Media, 2013.
- SCIKIT-LEARN. Documentação oficial. Disponível em: <https://scikit-learn.org>
- R CORE TEAM. R: A language and environment for statistical computing. R Foundation for Statistical Computing, Vienna, Austria. Disponível em: <https://www.R-project.org>
- KAGGLE. Credit Score Classification Dataset. Disponível em: <https://www.kaggle.com>
- PYTHON SOFTWARE FOUNDATION. Python Language Reference. Disponível em: <https://www.python.org>
- MALVEZZI, William. **Introdução à Ciência de Dados (Atualizado)**: Fundamentos de Inteligência Artificial. Brasília: Universidade Católica de Brasília, 2026. Slides apresentados na disciplina de Ciência de Dados.
- MALVEZZI, William. **Introdução aos Algoritmos Preditivos**: Fundamentos de Inteligência Artificial. Brasília: Universidade Católica de Brasília, 2026. Slides apresentados na disciplina de Ciência de Dados.
- SCIKIT-LEARN Developers. **Scikit-learn user guide**. version 1.4.2. 2024. Disponível em: <https://scikit-learn.org>. Acesso em: 20 mai. 2026